JAVA HYBRID INHERITANCE PROGRAM

Object-Oriented Programming Practical Record

1. Aim

To write a Java program using hybrid inheritance to perform bank account operations such as displaying balance, depositing money and withdrawing money.

2. Steps for Execution

1. Open VS Code, Eclipse or IntelliJ IDEA with JDK installed.
2. Create a Java file named **Main.java**.
3. Create the BankAccount parent class with the balance variable and showBalance() method.
4. Create the Deposit interface with the deposit() method.
5. Create the Withdraw interface with the withdraw() method.
6. Create SavingsAccount extending BankAccount and implementing Deposit and Withdraw.
7. Create CurrentAccount extending BankAccount and implementing Deposit.
8. Create objects for SavingsAccount and CurrentAccount.
9. Perform deposit and withdrawal operations.
10. Compile and run the program.

3. Algorithm

1. Start the program.
2. Create the BankAccount class and initialize balance = 10000.
3. Create the Deposit interface with deposit(double amount).
4. Create the Withdraw interface with withdraw(double amount).
5. Create SavingsAccount by extending BankAccount and implementing Deposit and Withdraw.
6. Create CurrentAccount by extending BankAccount and implementing Deposit.
7. Create a SavingsAccount object.
8. Display the initial balance, deposit 5000 and withdraw 2000.
9. Create a CurrentAccount object.
10. Display the initial balance and deposit 3000.
11. Stop the program.

4. Explanation of the Program

BankAccount class: It is the parent class containing the balance variable and the showBalance() method.

Deposit interface: It declares the deposit(double amount) method.

Withdraw interface: It declares the withdraw(double amount) method.

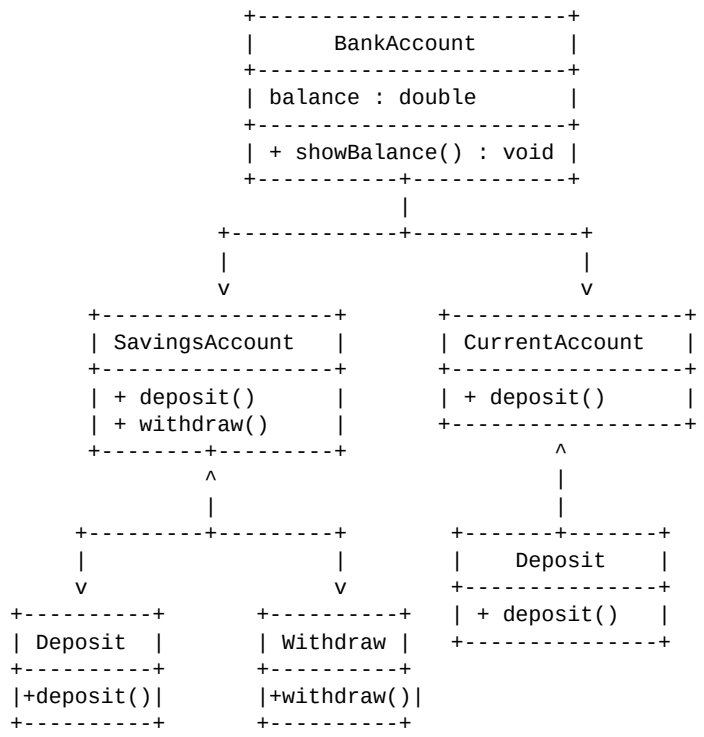
SavingsAccount class: It extends BankAccount and implements both Deposit and Withdraw interfaces. It performs deposit and withdrawal operations.

CurrentAccount class: It extends BankAccount and implements the Deposit interface. It performs the deposit operation.

Hybrid Inheritance: The program combines class inheritance using **extends** and multiple inheritance

through interfaces using **implements**.

5. UML Class Diagram



6. Java Source Code

```
// Hybrid Inheritance
// Real application: Bank Account

class BankAccount {

    double balance = 10000;

    void showBalance() {
        System.out.println("Initial Balance = " + balance);
    }
}

interface Deposit {

    void deposit(double amount);
}

interface Withdraw {

    void withdraw(double amount);
}

class SavingsAccount extends BankAccount implements Deposit, Withdraw {

    public void deposit(double amount) {
        balance = balance + amount;
        System.out.println("After Deposit = " + balance);
    }

    public void withdraw(double amount) {
        balance = balance - amount;
        System.out.println("After Withdrawal = " + balance);
    }
}

class CurrentAccount extends BankAccount implements Deposit {

    public void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Current Account Balance = " + balance);
    }
}

public class Main {
    public static void main(String[] args) {

        SavingsAccount savings = new SavingsAccount();

        savings.showBalance();
        savings.deposit(5000);
        savings.withdraw(2000);

        CurrentAccount current = new CurrentAccount();

        current.showBalance();
        current.deposit(3000);
    }
}
```

7. Output of the Program

```
Initial Balance = 10000.0
After Deposit = 15000.0
After Withdrawal = 13000.0
Initial Balance = 10000.0
Current Account Balance = 13000.0
```

8. Result

Thus, the Java program successfully demonstrates hybrid inheritance by combining inheritance through classes and interfaces to perform deposit, withdrawal and balance display operations for SavingsAccount and CurrentAccount.